

Cala's Pokecom BASIC System

Version 0.92

Programming Reference Manual /
プログラミング・リファレンスマニュアル

for ClockworkPi PicoCalc
with Raspberry Pi Pico 2 W

Copyright (C) 2026 Cala Maclir

目次

1. BASIC Language Overview	4
2. Lexical Elements	4
3. Data Types, Variables, Arrays	4
DIM	4
4. Operator Precedence	5
5. Assignment, PRINT, INPUT	5
LET / Assignment	5
PRINT	5
INPUT	5
6. Conditional and Loops	6
IF ... THEN ... ELSE	6
FOR ... NEXT	6
WHILE ... WEND	6
DO ... LOOP [UNTIL]	6
7. Branch and Termination (番号分岐はClassic専用)	6
8. Timing and Input	7
SLEEP	7
PAUSE	7
INKEY	7
TIMER	7
INKEY Key Code	7
9. Numeric Functions	8

10. String Functions	9
11. Graphics Reference	9
12. PCG Reference	10
GDEF	10
GPALETTE	10
13. Audio Reference	10
BEEP	10
PLAY	10
MML	11
WAV / MP3	11
14. I2C Reference	11
I2C SCAN	11
I2CREAD	11
I2CWRITE	12
15. Other BASIC Statements	12
16. REPL Command Reference	12
v0.92 storage / compile behavior	13
17. Runtime Limits and Errors	13
18. Quick Reference	14
Statements (alphabetical)	14
Functions (alphabetical)	14
19. Structured BASIC	14
FUNCTION / END FUNCTION / RETURN	14
20. 実装上限と非対応構文	16

for ClockworkPi PicoCalc with Raspberry Pi Pico 2 W

Copyright (C) 2026 Cala Maclir

1. BASIC Language Overview

CPB BASICは行番号付きClassic program、行番号なしStructured program、即時実行するDirect modeを持ちます。Structured programはEditorで作成・編集します。

```
10 PRINT "HELLO"  
20 END
```

- keywordとidentifierはcase-insensitiveです。
- statementはcolon (:) で区切れます。
- commentは **REM** またはapostrophe (') から行末までです。
- Classicのline numberは0〜2,147,483,647です。
- 同じline numberの入力は既存lineを置換します。line numberだけなら削除します。
- programの保存上限はProgram Storage backendに従います。PSRAM利用時のINTERNAL RAMとSDはともに1,024行・本文2,047 charactersです。PSRAMなしのINTERNAL SRAM fallbackは256行・本文191 charactersです。
- **EDIT**でFull-Screen BASIC Editorを開けます。PSRAM INTERNAL／SDの2,047文字lineはEditorまたはPCで編集できます。Direct modeと通常prompt入力191文字です。

2. Lexical Elements

要素	構文／範囲
decimal number	12、-3.5、.25
hexadecimal number	&HFF、&h20。0〜0x7fffffff
string literal	"TEXT"。quoteのescape syntaxはありません
identifier	英字または_で開始し、英数字・_・\$を続ける
string variable	末尾\$。例：NAME\$
separator	colon :
expression separator	comma ,
PRINT separator	comma , またはsemicolon ;

3. Data Types, Variables, Arrays

数値はsingle-precision floating pointとして扱います。文字列variableはidentifier末尾に\$を付けます。数値と文字列を混在して演算することはできません。ただし + は文字列連結にも使用します。

```
10 A=12.5  
20 NAME$="CPB"  
30 PRINT NAME$+" ";A
```

DIM

分類 Array Statement 構文 **DIM name(size[,size])[,name(size[,size])]**... 説明 1Dまたは2D arrayを確保します。string arrayはname末尾に\$を付けます。例

```
10 DIM A(99),M$(9,9)
```

制限

種類	合計上限
numeric array cells	4,096
string array cells	512
string array element	127 characters

添字とsizeは数値expressionです。Direct modeのarrayはcommand完了後にresetされます。

4. Operator Precedence

同じ優先順位のbinary operatorは左から評価されます。指数演算 ^ もVersion 0.92では左結合です。

優先順位（高→低）	operator
1	primary、function call、 parentheses
2	unary +、 -、 NOT
3	^
4	*, /、 MOD
5	+, -
6	=、 <>、 <、 <=、 >、 >=
7	AND
8	OR

comparisonの結果は数値です。logical operatorのoperandも数値でなければなりません。

5. Assignment、 PRINT、 INPUT

LET / Assignment

分類 Assignment Statement 構文 `[LET] variable=expression` 説明 scalar variableまたはarray elementへ代入します。LETは省略できます。

```
10 LET A=10
20 A$="PICO"
30 A(2)=A*2
```

PRINT

分類 Output Statement 構文 `PRINT [expression[;], expression]...` 説明 expressionをconsoleへ出力します。argumentなしは改行します。semicolonで後続出力を連結し、commaはtabulated fieldへ進めます。行末semicolonは改行しません。

```
10 PRINT "X=";12
20 PRINT "A", "B", "C"
30 PRINT SPC(3);"OK"
```

INPUT

分類 Input Statement 構文 `INPUT ["prompt";] variable` 説明 consoleから1個のscalar variableを読み込みます。variableの型に従って数値または文字列へ変換します。例 `INPUT "NAME";N$`

6. Conditional and Loops

IF ... THEN ... ELSE

分類 Conditional Statement 構文 **IF** expression **THEN** statement-or-line [**ELSE** statement-or-line] 説明 数値 expressionが真のときTHEN側、偽のときELSE側を実行します。THEN／ELSEの後にはline numberまたは1個の inline statementを置けます。

```
10 IF A>0 THEN PRINT "PLUS" ELSE PRINT "ZERO OR MINUS"
20 IF A=1 THEN 100
```

FOR ... NEXT

分類 Loop Statement 構文 **FOR** variable=start **TO** limit [**STEP** increment] / **NEXT** [variable] 説明 numeric loopです。STEP省略時は1です。制限 compile時のnesting上限は16です。

```
10 FOR I=1 TO 10 STEP 2
20 PRINT I
30 NEXT I
```

WHILE ... WEND

分類 Loop Statement 構文 **WHILE** expression / **WEND** 説明 expressionが真の間、bodyを繰り返します。制限 compile時のnesting上限は16です。

DO ... LOOP [UNTIL]

分類 Loop Statement 構文 **DO** / **LOOP** [**UNTIL** expression] 説明 UNTILなしでは無限loop、UNTILありでは expressionが真になるまで繰り返します。制限 compile時のnesting上限は16です。

```
10 DO
20 K=INKEY
30 LOOP UNTIL K=27
```

7. Branch and Termination（番号分岐はClassic専用）

名称	分類	構文	説明
GOTO	Branch	GOTO line	指定lineへ移動
GOSUB	Subroutine	GOSUB line	subroutineを呼び出す
RETURN	Subroutine	RETURN	GOSUBの呼出元へ戻る
ON GOTO	Branch	ON expression GOTO line[,line...]	1始まりのselectorで移動先を選択
ON GOSUB	Subroutine	ON expression GOSUB line[,line...]	1始まりのselectorでsubroutineを選択
END	Termination	END	programを通常終了
STOP	Termination	STOP	programを停止

BREAKは実行中programを停止し、background audioも停止します。END、STOP、通常のruntime errorは background audioを自動停止しません。

8. Timing and Input

SLEEP

分類 Timing Statement 構文 **SLEEP milliseconds** 説明 指定時間待機します。background serviceを継続し、BREAKを受け付けます。例 **SLEEP 100**

PAUSE

分類 Input Statement 構文 **PAUSE** 説明 画面を変更せず通常keyを待ちます。

INKEY

分類 Numeric Function 構文 **INKEY** または **INKEY()** 戻り値 入力なしは0、入力ありはkey code。説明 non-blockingでkeyを読みます。

TIMER

分類 Numeric Function 構文 **TIMER** または **TIMER()** 戻り値 system startからの経過時間。

INKEY Key Code

key	code
Enter	0x0A
Carriage Return	0x0D
Escape	0xB1
Left	0xB4
Up	0xB5
Down	0xB6
Right	0xB7
Home	0xD2
Delete	0xD4
printable ASCII	ASCII code

physical keyboardとBluetooth HID keyboardはCPBの共通input code体系へ変換されます。

9. Numeric Functions

名称	構文	戻り値・説明
ABS	ABS(x)	absolute value
INT	INT(x)	integer directionへの変換
VAL	VAL(string\$)	stringを数値へ変換
RND	RND / RND()	pseudo-random number
RNDI	RNDI(limit)	integer random value
SIN	SIN(x)	sine
COS	COS(x)	cosine
TAN	TAN(x)	tangent
SQR	SQR(x)	square root
ATN	ATN(x)	arctangent
LOG	LOG(x)	natural logarithm
EXP	EXP(x)	exponential
PI	PI / PI()	π
RAD	RAD(degrees)	degreeをradianへ変換
DEG	DEG(radians)	radianをdegreeへ変換
SGN	SGN(x)	sign (-1、0、1)
MIN	MIN(a,b)	小さい方
MAX	MAX(a,b)	大きい方
CLAMP	CLAMP(value,low,high)	rangeへclamp
TIMER	TIMER / TIMER()	elapsed timer
INKEY	INKEY / INKEY()	non-blocking key input
I2CREAD	I2CREAD(address,register)	I2C read value
PLAYING	PLAYING()	PLAYがactiveなら真
POINT	POINT(x,y)	graphics pixelがnon-blackなら真

functionを式として使用する場合、zero-argument function以外はparenthesesが必要です。

```
10 A=MAX(ABS(-5),INT(2.9))
20 PRINT SIN(RAD(90))
```

10. String Functions

名称	構文	戻り値・説明
STR\$	STR\$(number)	数値の文字列表現
LEN	LEN(string\$)	character数
CHRS	CHRS(code)	codeから1文字を作る
ASC	ASC(string\$)	先頭文字のcode
LEFT\$	LEFT\$(string\$,length)	左からlength文字
RIGHT\$	RIGHT\$(string\$,length)	右からlength文字
MID\$	MID\$(string\$,start[,length])	startから部分文字列
INSTR	INSTR(string\$,find\$)	部分文字列の位置
STRING\$	STRING\$(count,string\$)	stringをcount回反復
SPC	SPC(count)	count個のspace
TAB	TAB(column)	PRINT用tabulation string

```
10 A$=MID$("PICOALC",5,4)
20 PRINT LEFT$(A$,2);STRING$(3,"!")
```

11. Graphics Reference

physical LCDは320×320 pixelです。virtual coordinateはSCREENで定義し、縦横比を保ってLCD中央へmappingされます。

名称	構文	説明
SCREEN	SCREEN [width,height]	virtual screenを設定。引数なしは640×480
CLS	CLS	graphics screenをclear
COLOR	COLOR palette / COLOR r,g,b	drawing colorを設定
COLORHSV	COLORHSV h,s,v	HSVからdrawing colorを設定
PSET	PSET x,y[,color]	pixelを描く
LINE	LINE x1,y1,x2,y2[,color]	lineを描く
LINE	LINE -x,y[,color]	current graphics pointからlineを描く
CIRCLE	CIRCLE x,y,r[,color]	circleを描く
BOX	BOX x1,y1,x2,y2[,filled[,color]]	rectangleを描く
PAINT	PAINT x,y[,color]	closed regionをflood fill
POINT	POINT(x,y)	pixel query
FLUSH	FLUSH	graphics bufferをLCDへ反映
GLOCATE	GLOCATE x,y	graphics text cursorを設定
GPRINT	GPRINT expression[,expression...]	graphics textを描く
SAVEIMAGE	SAVEIMAGE "name"[,x1,y1,x2,y2]	BMP保存
SAVE IMAGE	SAVE IMAGE "name"[,x1,y1,x2,y2]	SAVEIMAGEと同義

COLORのRGB componentは0～255、COLORHSVのhはdegree、s/vは0～1です。PAINTはfill対象が確定しない場合、または領域が大きい場合に時間を要します。

```

10 SCREEN 320,320:CLS
20 COLOR 255,255,255
30 LINE 0,0,319,319
40 CIRCLE 160,160,80
50 FLUSH

```

12. PCG Reference

GDEF

分類 Graphics Statement 構文 [GDEF "character"="hexdata"](#) / [GDEF "character"=""](#) / [GDEF CLEAR](#) 説明
printable ASCII 0x20~0x7Eに8×8 PCG glyphを定義します。

mono PCGは16 hex digits (8 rows×1 byte) です。bit 7がleft pixelです。set bitはcurrent COLOR、clear bitはbackgroundを描きます。

```

10 GDEF "A"="183C66667E666600"
20 GLOCATE 20,20:GPRINT "A"

```

128 hex digitsでは64個のpalette index byteによるindexed-color PCGです。index 0はtransparentです。

GPALETTE

分類 Graphics Statement 構文 [GPALETTE index,r,g,b](#) / [GPALETTE index,rgb24](#) / [GPALETTE RESET](#) 説明
indexed-color PCGのpaletteを設定します。indexは1~255です。

```

10 GPALETTE 8,18,38,84
20 GPALETTE 9,&H1ED6FF
30 GPALETTE RESET

```

13. Audio Reference

BEEP

分類 Audio Statement 構文 [BEEP frequency_hz,duration_ms](#) 範囲 frequency 20~20,000 Hz、duration 1~60,000 ms。例 [BEEP 880,150](#)

PLAY

分類 Audio Statement 構文

```

PLAY voice1$[,voice2$[,voice3$]]
PLAY PAUSE
PLAY RESUME
PLAY WAIT
PLAY STOP
PLAYING()

```

最大3 voiceです。PLAYはbackgroundで開始します。PLAY WAITは完了まで待機します。PLAY STOPはMML playbackを停止します。

MML

token	意味
Tn	tempo、32～400
On	octave、0～8
Ln	default note length。1、2、4、8、16、32
Vn	level、0～15
C D E F G A B	note
R	rest
# / +	sharp
-	flat
< / >	octave down / up
number	note length
.	dotted note

MMLは1 voiceあたり384 charactersまでです。

```
10 PLAY "T12005L8V12 CDEFGAB>C", "T12004L4V8 CEGC"
20 PLAY WAIT
```

WAV / MP3

名称	構文	説明
WAVPLAY	WAVPLAY "file"	background WAV / MP3 playback開始
WAVPAUSE	WAVPAUSE	playbackをpause
WAVRESUME	WAVRESUME	playbackをresume
WAVSTOP	WAVSTOP	playbackを停止

file内容からformatを判定します。対応formatは次のとおりです。

- RIFF/WAVE PCM（format 1）、8/16-bit、mono/stereo、11,025／22,050／44,100 Hz
- MP3、44,100／48,000 Hz、mono/stereo

compressed WAV、24-bit WAV、48 kHz WAV、対応外sample rateのMP3は対象外です。MP3再生時は必要に応じてCPU 200 MHzへ自動移行し、停止後に元のprofileへ戻ります。

BREAKはPLAY/WAVを停止します。END、STOP、通常のruntime error後もbackground audioは継続します。

14. I2C Reference

I2C SCAN

分類 I2C Statement 構文 [I2C SCAN](#) 説明 7-bit addressをscanします。

I2CREAD

分類 Numeric Function 構文 [I2CREAD\(address,register\)](#) 説明 external I2Cから1 byteをreadします。

I2CWRITE

分類 I2C Statement 構文 `I2CWRITE address,register,value` 範囲 address 0x08~0x77、register 0~255、value 0~255。

```
10 I2C SCAN
20 V=I2CREAD(&H51,&H02)
30 I2CWRITE &H20,&H01,&HFF
```

15. Other BASIC Statements

名称	構文	説明
LOCATE	LOCATE column,row	console text cursor位置を設定
RANDOMIZE	RANDOMIZE [seed]	pseudo-random sequenceを初期化
REM	REM text	comment
'	' text	comment

16. REPL Command Reference

以下はBASIC statementではなく、`BASIC>` promptで実行するREPL commandです。

command	構文	説明
EDIT	EDIT	Full-Screen BASIC Editorを開く
LIST	LIST	current programを表示。BREAK/Esc/Ctrl-Cで途中停止
RUN	RUN	current programを実行。開始時にRUN...を表示
NEW	NEW	program、Direct scalar、current filenameをclear
CLEAR	CLEAR	Direct scalarをclear
CLS	CLS	console/graphics画面をclear
LOAD	LOAD "name"	BASIC sourceをload。開始時にLOADING... (filename)を表示
SAVE	SAVE / SAVE "name"	current fileへsave/Save As
FILES	FILES ["directory"]	SDのBASIC一覧。省略時はroot、指定時は相対directory
DIR	DIR ["directory"]	SDのfile/directory一覧
SD	SD / SD STATUS / SD REMOUNT	SD status/remount
SCREENSHOT	SCREENSHOT ["name"]	LCD BMPをSDへ保存
XRECV	XRECV "name"	XMODEM receive
XSEND	XSEND "name"	XMODEM send
YRECV	YRECV	YMODEM receive
YSEND	YSEND "name"	YMODEM send
DATE	DATE [yyyy-mm-dd]	date表示/設定
TIME	TIME [hh:mm:ss]	time表示/設定
DATETIME	DATETIME	dateとtimeを表示
SERIAL	SERIAL ...	serial route設定表示/変更

command	構文	説明
CONSOLE	CONSOLE LCD ONLY等	console mode変更
PROFILE	PROFILE ...	execution profile表示／制御
STANDBY	STANDBY	RAM保持待機
MENU	MENU	Control Centerを開く
HELP	HELP	command概要

SAVE IMAGE はBASIC statement、**SAVE** はREPL commandです。**EDIT** はprogram内には記述できません。

v0.92 storage / compile behavior

Stored programの初回**RUN**ではsourceをcompileし、PSRAMが利用できる場合はCompiled Program cacheへ保存します。同じsourceを再度**RUN**するとcacheを復元して内部SRAMのexecution workspaceへ展開します。VMはPSRAM上のopcodeを直接実行しません。sourceを編集、LOAD、NEW、storage mode切替した場合はrevisionが変化し、古いcacheは再利用されません。

Direct scalar stateもPSRAMへ保存できますが、Direct statement実行時は内部SRAM working copyへ復元して処理します。これらは言語仕様を変更せず、runtime memory使用量と再compile overheadを改善する実装です。

17. Runtime Limits and Errors

resource	limit
compiled operations	1,536
symbols	64
compiled line map	inline 256 + extended map up to program line count
FOR nesting	16
WHILE nesting	16
DO nesting	16
numeric array cells	4,096
string array cells	512
string array element	127 characters
MML per voice	384 characters
INTERNAL program / PSRAM	1,024 lines / 2,047 body chars
INTERNAL program / SRAM fallback	256 lines / 191 body chars
SD program	1,024 lines / 2,047 body chars
Direct / prompt input line	191 body chars

代表的なerror :

category	例	対処
Compile	SYNTAX ERROR、TYPE MISMATCH	syntaxとdata typeを確認
Control flow	NEXT WITHOUT FOR、WEND WITHOUT WHILE	pairを確認
Resource	PROGRAM TOO COMPLEX、TOO MANY VARIABLES	programを分割、variableを削減

category	例	対処
Storage	LINE TOO LONG FOR INTERNAL PSRAM、 LINE TOO LONG FOR SRAM FALLBACK、 PROGRAM TOO LARGE FOR INTERNAL MODE	current backend capacityを確認。PSRAM利 用時のINTERNALは1024×2047
I/O	transfer timeout、SD mount failure	route、cable、SD状態を確認

18. Quick Reference

Statements (alphabetical)

BEEP、BOX、CIRCLE、CLS、COLOR、COLORHSV、DIM、DO、END、FLUSH、FOR、GDEF、GLOCATE、
GOSUB、GOTO、GPALETTE、GPRINT、I2C SCAN、I2CWRITE、IF、INPUT、LET、LINE、LOCATE、LOOP、
NEXT、ON ... GOSUB、ON ... GOTO、PAINT、PAUSE、PLAY、PRINT、PSET、RANDOMIZE、REM、RETURN、
SAVE IMAGE、SAVEIMAGE、SCREEN、SLEEP、STOP、WAVPAUSE、WAVPLAY、WAVRESUME、WAVSTOP、
WEND、WHILE

Functions (alphabetical)

ABS、ASC、ATN、CHR\$, CLAMP、COS、DEG、EXP、I2CREAD、INKEY、INSTR、INT、LEFT\$, LEN、LOG、
MAX、MID\$, MIN、PI、PLAYING、POINT、RAD、RND、RNDI、RIGHT\$, SGN、SIN、SPC、SQR、STR\$,
STRING\$, TAB、TAN、TIMER、VAL

実行可能なより大きいsampleはrepositoryの [examples/](#) を参照してください。

19. Structured BASIC

LOADは空行を除く全行に番号があればClassic、全行に番号がなければStructuredと判定します。混在はMIXED
SOURCE MODEで拒否し、現在sourceを保持します。空／空行だけのfileは現在sessionのmodeを保持します。
Structured保存は行番号を付けず、空行・indent・本文を保持します（改行はLF）。LISTも番号なしです。
StructuredではGOTO、GOSUB、ON GOTO/GOSUBを使用できません。Direct mode、Classicの番号入力／分岐
／RETURNは従来どおりです。

THEN直後が行末なら複数行IFです。statementが続けば従来のsingle-line IFです。

```
SCORE=75
IF SCORE>=100 THEN
    PRINT "CLEAR"
ELSEIF SCORE>=50 THEN
    PRINT "GOOD"
ELSE
    PRINT "TRY AGAIN"
END IF
```

ELSEIFは複数、ELSEは省略可能です。IFは入れ子にでき、FOR/NEXT、WHILE/WEND、DO/LOOP、DO
WHILE/LOOP、DO/LOOP UNTILと組み合わせられます。blockの欠落／二重ELSE／ELSE後ELSEIF／loop境界の
交差はcompile errorです。keywordは大文字小文字を区別せず、indentは任意です。

FUNCTION / END FUNCTION / RETURN

```

PRINT SQUARE(12)
FUNCTION SQUARE(X)
    RETURN X*X
END FUNCTION

FUNCTION WRAP$(S$)
    RETURN "["+S$+"]"
END FUNCTION
PRINT WRAP$("CPB")

```

FUNCTIONはStructured top-levelだけに置けます。定義より前から呼べます。definition bodyはmain実行で飛び越します。名前末尾\$は文字列返値、その他は数値返値です。parameterも同じ型規則で値渡しです。括弧はzero-argument呼出しにも必要です。argument数／型とRETURN expressionの型はcompile時に検査します。

数値localの初期値は0、文字列localは空です。parameter、代入先、未宣言の参照は関数localとなり、関数ごと／呼出しごとに独立します。同名globalを暗黙に参照しません。

```

SCORE=0
FUNCTION ADD_SCORE(POINT)
    GLOBAL SCORE
    SCORE=SCORE+POINT
    RETURN SCORE
END FUNCTION
PRINT ADD_SCORE(10)
PRINT ADD_SCORE(20)
PRINT SCORE

```

GLOBALはFUNCTION body先頭の実行statementより前に宣言し、read/write両方へ適用します。複数名はGLOBAL SCORE,NAME\$のようにcommaで区切ります。commentと空行は実行statementではありません。parameter/localとGLOBALの同名は拒否します。関数内arrayは非対応です。

RETURN expressionはFUNCTION内専用です。数値／文字列の型は一致させてください。RETURNなし、expressionなしはcompile errorです。RETURNを通らずEND FUNCTIONに到達するとFUNCTION RETURN MISSINGになります。Classic RETURNはGOSUBから戻る既存動作を維持します。

関数名はcase-insensitiveで、FとF\$は同一base nameです。duplicate、builtinとのbase-name衝突、同base-name scalarとの曖昧性は拒否します。Structured identifierは最大16文字です。

制限	値
FUNCTION数	32
parameter数／関数	8
scalar local数／関数（parameterを含む）	128
nested call／recursion depth	16
string local／parameter／return	127文字
global symbol数	64

再帰を利用できます。17段目はFUNCTION CALL DEPTH、SRAM不足はOUT OF MEMORYとなります。大きいstring frameでは16段より前にSRAM不足となる場合があります。BREAK／runtime error／通常return後にframeは解放されます。

```
FUNCTION FACT(N)
  IF N<=1 THEN
    RETURN 1
  END IF
  RETURN N*FACT(N-1)
END FUNCTION
PRINT FACT(5)
```

期待値は120です。Structured errorはAT ROWを表示し、関数内ではIN FUNCTION、CALLED FROM ROWとDEPTH也表示します。ROWはsource中の位置であり、BASICの分岐行番号ではありません。Classic errorのIN 行番号は従来どおりです。

20. 実装上限と非対応構文

項目	v0.92の制限
IL命令	1,536 slots
literal string pool	6,144 bytes
global symbol	64
identifier	最大16文字
source容量	PSRAM INTERNAL／SDは1024×2047、SRAM fallbackは256×191
prompt／Direct入力	191文字

ソース容量内でもIL／pool上限に達する場合があります。BAS保存はLF改行です。パスはSDルート相対で区切りを含め79文字です。PSRAMなしではEditor履歴を利用できません。

FUNCTION内のlocal配列、GLOBAL配列アクセス、SUB、labels、STATIC、optional parameter、overload、moduleは非対応です。DATA／READ／RESTORE、INPUT強化、BASIC File I/O、TRACE／Debuggerはv0.93候補です。OTAとBASICネイティブコンパイラは含みません。

v0.92は引き続きVMでBASICを実行します。FUNCTION有無に応じたVM内部C++実装の選択と融合命令で処理を改善しています。single precisionと丸め規則を維持し、新local数値helperではFMA contractionを禁止しています。PROFILE COUNTは機構確認用、通常速度はPROFILE OFFで測定してください。RUNの表示時間はVM実行で、compile-onlyやcache-onlyの時間を示しません。